

IMPORTS ET CONFIGURATION DE BASE

DESCRIPTION : Imports standards et configuration initiale de numpy
 POURQUOI : Nécessaire avant toute utilisation de numpy.
 CONTEXTE : Début de tout script scientifique ou analytique.

IMPORTS STANDARDS :

import numpy as np	Import principal
import numpy.linalg as la	Algèbre linéaire
import numpy.fft as fft	Transformées de Fourier
import numpy.random as rng	Génération aléatoire (ancienne API)
from numpy.random import default_rng	Générateur moderne (recommandé)
import numpy.ma as ma	Tableaux masqués
import numpy.polynomial as poly	Polynômes
import numpy.testing as npt	Tests unitaires numpy

INFORMATIONS :

np.__version__	Version de numpy
np.show_config()	Affiche la configuration de compilation
np.get_include()	Chemin des headers C numpy

OPTIONS D'AFFICHAGE :

np.set_printoptions(precision=4)	Nombre de décimales affichées
np.set_printoptions(suppress=True)	Supprime la notation scientifique
np.set_printoptions(threshold=100)	Nb d'éléments avant troncature
np.set_printoptions(threshold=np.inf)	Affiche tout (pas de troncature)
np.set_printoptions(linewidth=120)	Largeur de ligne
np.set_printoptions(edgeitems=5)	Éléments affichés aux bords
np.set_printoptions(formatter={"float": "{:.2f}".format})	Format personnalisé
np.get_printoptions()	Retourne les options courantes
with np.printoptions(precision=2):	Options temporaires
print(arr)	

[np] Création de tableaux – array, zeros, ones, arange, linspace...

DESCRIPTION : Crée des tableaux numpy depuis diverses sources
 POURQUOI : Point de départ de tout calcul numérique avec numpy.
 CONTEXTE : Initialisation de données, génération de grilles, tests.

— DEPUIS DES DONNÉES EXISTANTES —

np.array([1, 2, 3])	Depuis une liste Python
np.array([[1,2],[3,4]])	Depuis une liste 2D (matrice)
np.array([1,2,3], dtype=float)	Type forcé
np.array([1,2,3], dtype=np.float64)	Type numpy explicite
np.array([1,2,3], dtype=np.int32)	Entier 32 bits
np.array([1,2,3], dtype=np.complex128)	Complexe
np.array([1,2,3], dtype="U10")	Chaîne unicode 10 chars
np.array([1,2,3], copy=True)	Copie explicite (défaut)
np.array([1,2,3], copy=False)	Pas de copie si possible
np.array([1,2,3], ndmin=2)	Dimension minimale
np.array([1,2,3], order="C")	Ordre ligne (C-contiguous)
np.array([1,2,3], order="F")	Ordre colonne (Fortran-contiguous)
np.asarray([1,2,3])	Convertit sans copier si possible
np.asarray(liste, dtype=float)	Convertit en float
np.ascontiguousarray(arr)	Rend C-contiguous
np.asfortranarray(arr)	Rend F-contiguous
np.frombuffer(buffer, dtype=float)	Depuis un buffer mémoire
np.fromstring("1 2 3", sep=" ")	Depuis une chaîne
np.fromiter(iterable, dtype=float)	Depuis un itérateur
np.fromfile("data.bin", dtype=float)	Depuis un fichier binaire
np.loadtxt("data.txt")	Depuis un fichier texte
np.loadtxt("data.txt", delimiter=",")	Avec délimiteur
np.loadtxt("data.txt", usecols=(0,2))	Colonnes spécifiques
np.loadtxt("data.txt", skiprows=1)	Saute la première ligne
np.loadtxt("data.txt", dtype=str)	En chaînes
np.genfromtxt("data.csv", delimiter=",")	Gère les valeurs manquantes
np.genfromtxt("data.csv", names=True)	Utilise la première ligne comme noms
np.genfromtxt("data.csv", filling_values=0)	Remplace les NaN

<code>np.genfromtxt("data.csv", dtype=None)</code>	Inférence automatique des types
<code>np.genfromtxt("data.csv", encoding="utf-8")</code>	Encodage spécifique

— TABLEAUX CONSTANTS —

<code>np.zeros(5)</code>	Vecteur de 5 zéros
<code>np.zeros((3, 4))</code>	Matrice 3x4 de zéros
<code>np.zeros((2, 3, 4))</code>	Tenseur 3D
<code>np.zeros(5, dtype=int)</code>	Zéros entiers
<code>np.zeros_like(arr)</code>	Zéros de même shape/dtype que arr
<code>np.ones(5)</code>	Vecteur de uns
<code>np.ones((3, 4))</code>	Matrice de uns
<code>np.ones_like(arr)</code>	Uns de même shape/dtype
<code>np.full(5, 7.0)</code>	Vecteur rempli de 7.0
<code>np.full((3, 4), np.pi)</code>	Matrice remplie de π
<code>np.full_like(arr, 42)</code>	Rempli de même shape/dtype
<code>np.empty(5)</code>	Tableau non initialisé (rapide)
<code>np.empty((3, 4))</code>	Matrice non initialisée
<code>np.empty_like(arr)</code>	Non initialisé de même shape/dtype

— TABLEAUX IDENTITÉ ET DIAGONAUX —

<code>np.eye(3)</code>	Matrice identité 3x3
<code>np.eye(3, 4)</code>	Matrice identité 3x4
<code>np.eye(3, k=1)</code>	Diagonale décalée de 1
<code>np.eye(3, dtype=int)</code>	En entiers
<code>np.identity(3)</code>	Matrice identité carrée
<code>np.diag([1, 2, 3])</code>	Matrice diagonale depuis un vecteur
<code>np.diag(matrice)</code>	Extrait la diagonale d'une matrice
<code>np.diag(matrice, k=1)</code>	Diagonale décalée
<code>np.diagflat([1,2,3])</code>	Diagonale depuis un tableau aplati
<code>np.tri(3)</code>	Matrice triangulaire inférieure
<code>np.tri(3, k=0)</code>	Avec décalage
<code>np.tril(matrice)</code>	Triangulaire inférieure
<code>np.triu(matrice)</code>	Triangulaire supérieure
<code>np.tril(matrice, k=-1)</code>	Sans la diagonale principale

— TABLEAUX DE SÉQUENCES —

<code>np.arange(10)</code>	[0, 1, ..., 9]
<code>np.arange(2, 10)</code>	[2, 3, ..., 9]
<code>np.arange(0, 1, 0.1)</code>	[0.0, 0.1, ..., 0.9]
<code>np.arange(10, dtype=float)</code>	En flottants
<code>np.linspace(0, 1, 100)</code>	100 points entre 0 et 1 (inclus)
<code>np.linspace(0, 1, 100, endpoint=False)</code>	Sans inclure 1
<code>np.linspace(0, 1, 100, retstep=True)</code>	Retourne aussi le pas
<code>np.logspace(0, 3, 4)</code>	[1, 10, 100, 1000] (base 10)
<code>np.logspace(0, 3, 4, base=2)</code>	Base 2
<code>np.logspace(0, 3, 4, endpoint=False)</code>	Sans inclure la dernière
<code>np.geomspace(1, 1000, 4)</code>	Progression géométrique
<code>np.geomspace(1, 1000, 4, endpoint=False)</code>	Sans inclure la dernière

— GRILLES ET MESHGRID —

<code>np.meshgrid(x, y)</code>	Grille 2D depuis deux vecteurs
<code>np.meshgrid(x, y, z)</code>	Grille 3D
<code>np.meshgrid(x, y, indexing="ij")</code>	Indexation matricielle (i,j)
<code>np.meshgrid(x, y, indexing="xy")</code>	Indexation cartésienne (défaut)
<code>np.meshgrid(x, y, sparse=True)</code>	Matrices creuses (économie mémoire)
<code>np.mgrid[0:5, 0:5]</code>	Grille complète via slice
<code>np.mgrid[0:1:10j, 0:1:10j]</code>	Grille 10x10 entre 0 et 1
<code>np.ogrid[0:5, 0:5]</code>	Grille ouverte (sparse)
<code>np.indices((3, 4))</code>	Indices de chaque position
<code>np.ix_([0,1,2], [0,1])</code>	Indices ouverts pour broadcasting

— TABLEAUX SPÉCIAUX —

<code>np.nan</code>	Valeur Not a Number (float)
<code>np.inf</code>	Infini positif
<code>np.NINF</code>	Infini négatif
<code>np.PZERO</code>	Zéro positif
<code>np.NZERO</code>	Zéro négatif
<code>np.pi</code>	$\pi = 3.14159...$
<code>np.e</code>	$e = 2.71828...$
<code>np.euler_gamma</code>	Constante d'Euler $\gamma \approx 0.5772$
<code>np.newaxis</code>	Alias de None (ajout de dimension)

EXEMPLES :

```
x = np.linspace(-np.pi, np.pi, 1000)
X, Y = np.meshgrid(np.linspace(-5,5,100), np.linspace(-5,5,100))
```

```
Z = np.sin(np.sqrt(X**2 + Y**2))
```

[arr] Attributs et inspection

DESCRIPTION : Inspecte la structure et les métadonnées d'un tableau
POURQUOI : Comprendre la forme, le type et la taille des données.
CONTEXTE : Débogage, vérification, EDA.

ATTRIBUTS FONDAMENTAUX :

arr.shape	Tuple des dimensions (n, m, ...)
arr.ndim	Nombre de dimensions
arr.size	Nombre total d'éléments
arr.dtype	Type des éléments
arr.itemsize	Taille en octets d'un élément
arr.nbytes	Taille totale en octets
arr.T	Transposée
arr.real	Partie réelle (complexes)
arr.imag	Partie imaginaire (complexes)
arr.flat	Itérateur aplati
arr.flags	Informations de mémoire (C_CONTIGUOUS, etc.)
arr.strides	Pas en octets pour chaque dimension
arr.base	Tableau de base (si vue)
arr.data	Buffer mémoire des données

INSPECTION :

arr.dtype.name	Nom du type ("float64", "int32"...)
arr.dtype.kind	Catégorie ("f"=float, "i"=int, "U"=str...)
arr.dtype.itemsize	Taille en octets du dtype
arr.dtype.byteorder	Ordre des octets ("=" natif)
np.isscalar(val)	Vrai si scalaire
np.ndim(arr)	Nombre de dimensions
np.shape(arr)	Forme du tableau
np.size(arr)	Nombre d'éléments
arr.nbytes / 1024**2	Taille en Mo

EXEMPLES :

```
print(f"Shape: {arr.shape}, dtype: {arr.dtype}, size: {arr.nbytes/1024:.1f} KB")
```

[arr] Types de données – dtype, astype, casting

DESCRIPTION : Gère les types numériques de numpy
POURQUOI : Contrôler la précision, l'utilisation mémoire et la compatibilité.
CONTEXTE : Optimisation mémoire, interfaçage avec d'autres bibliothèques.

TYPES ENTIERS :

np.int8	Entier signé 8 bits (-128 à 127)
np.int16	Entier signé 16 bits
np.int32	Entier signé 32 bits
np.int64	Entier signé 64 bits (défaut)
np.uint8	Entier non signé 8 bits (0-255)
np.uint16	Entier non signé 16 bits
np.uint32	Entier non signé 32 bits
np.uint64	Entier non signé 64 bits
np.intp	Entier pour indexation (ptr size)

TYPES FLÔTANTS :

np.float16	Demi-précision (16 bits)
np.float32	Précision simple (32 bits)
np.float64	Double précision (64 bits, défaut)
np.float128	Précision étendue (si disponible)
np.longdouble	Long double (platform-dependent)

TYPES COMPLEXES :

np.complex64	Complexe 2×float32
np.complex128	Complexe 2×float64 (défaut)
np.complex256	Complexe 2×float128 (si disponible)

AUTRES TYPES :

np.bool_	Booléen numpy
np.str_	Chaîne unicode numpy
np.bytes_	Bytes numpy
np.object_	Objet Python arbitraire
np.void	Type vide (pour structured arrays)

LIMITES DES TYPES :

<code>np.iinfo(np.int32).max</code>	Valeur max d'un int32
<code>np.iinfo(np.int32).min</code>	Valeur min d'un int32
<code>np.finfo(np.float64).max</code>	Valeur max d'un float64
<code>np.finfo(np.float64).min</code>	Valeur min positive d'un float64
<code>np.finfo(np.float64).eps</code>	Epsilon machine (précision)
<code>np.finfo(np.float64).resolution</code>	Résolution décimale

CONVERSION DE TYPE :

<code>arr.astype(float)</code>	Convertit en float64
<code>arr.astype(np.int32)</code>	Convertit en int32
<code>arr.astype("U10")</code>	Convertit en chaîne unicode
<code>arr.astype(float, copy=False)</code>	Sans copie si déjà le bon type
<code>arr.astype(float, casting="safe")</code>	Conversion sûre seulement
<code>arr.astype(float, casting="same_kind")</code>	Même famille de type
<code>arr.view(np.float32)</code>	Vue avec un autre type (sans copie)
<code>np.can_cast(np.int32, np.float64)</code>	Teste si la conversion est possible
<code>np.result_type(np.int32, np.float64)</code>	Type résultant d'une opération mixte
<code>np.common_type(arr1, arr2)</code>	Type commun minimal

`[arr]` Manipulation de forme – reshape, resize, flatten, ravel, transpose

DESCRIPTION : Modifie la forme d'un tableau sans changer ses données
POURQUOI : Adapter la structure des données aux opérations requises.
CONTEXTE : Algèbre linéaire, deep learning, traitement d'images.

RESHAPE ET RESIZE :

<code>arr.reshape(3, 4)</code>	Redéfinit la forme (retourne une vue)
<code>arr.reshape(3, -1)</code>	-1 = dimension inférée automatiquement
<code>arr.reshape(-1)</code>	Aplatit en vecteur (comme ravel)
<code>arr.reshape((2, 3, 4))</code>	Tenseur 3D
<code>arr.reshape(3, 4, order="C")</code>	Ordre ligne (défaut)
<code>arr.reshape(3, 4, order="F")</code>	Ordre colonne (Fortran)
<code>np.reshape(arr, (3, 4))</code>	Fonction top-level
<code>arr.resize((3, 4))</code>	Modifie en place (peut copier)
<code>np.resize(arr, (3, 4))</code>	Copie avec répétition si nécessaire

APLATISSEMENT :

<code>arr.flatten()</code>	Copie aplatie en vecteur 1D
<code>arr.flatten(order="C")</code>	Ordre ligne (défaut)
<code>arr.flatten(order="F")</code>	Ordre colonne
<code>arr.ravel()</code>	Vue aplatie (pas de copie si possible)
<code>arr.ravel(order="C")</code>	Ordre ligne
<code>arr.ravel(order="F")</code>	Ordre colonne
<code>np.ravel(arr)</code>	Fonction top-level

TRANSPOSITION :

<code>arr.T</code>	Transposée (vue)
<code>arr.transpose()</code>	Inverse toutes les dimensions
<code>arr.transpose(1, 0, 2)</code>	Permute les dimensions (axe 1→0, 0→1...)
<code>np.transpose(arr)</code>	Fonction top-level
<code>np.transpose(arr, axes=(1, 0, 2))</code>	Avec permutation
<code>np.swapaxes(arr, 0, 1)</code>	Échange deux axes
<code>np.moveaxis(arr, 0, -1)</code>	Déplace l'axe 0 en dernier
<code>np.moveaxis(arr, [0,1], [-1,-2])</code>	Déplace plusieurs axes
<code>np.rollaxis(arr, 2, 0)</code>	Roule l'axe 2 vers la position 0

AJOUT / SUPPRESSION DE DIMENSIONS :

<code>arr[np.newaxis, :]</code>	Ajoute dimension en position 0
<code>arr[:, np.newaxis]</code>	Ajoute dimension en position 1
<code>arr[None, :]</code>	Identique à <code>np.newaxis</code>
<code>np.expand_dims(arr, axis=0)</code>	Ajoute une dimension en 0
<code>np.expand_dims(arr, axis=(0, 2))</code>	Ajoute plusieurs dimensions
<code>np.squeeze(arr)</code>	Supprime les dimensions de taille 1
<code>np.squeeze(arr, axis=0)</code>	Supprime seulement l'axe 0
<code>arr.squeeze()</code>	Méthode équivalente

EXEMPLES :

```
# Vecteur → matrice
v = np.arange(12)
m = v.reshape(3, 4)
# Batch de vecteurs → batch de matrices
X = X.reshape(X.shape[0], -1) # (N, H, W) → (N, H*W)
```

```
# Ajouter dimension batch
img = img[np.newaxis, ...]          # (H, W, C) → (1, H, W, C)
```

[arr] Indexation et slicing

DESCRIPTION : Accède à des sous-ensembles d'un tableau
 POURQUOI : Extraire, modifier et analyser des parties spécifiques des données.
 CONTEXTE : Filtrage, extraction de features, fenêtres glissantes.

— INDEXATION BASIQUE —

arr[0]	Premier élément
arr[-1]	Dernier élément
arr[2, 3]	Élément à la ligne 2, colonne 3
arr[2][3]	Identique (moins efficace)
arr[0, 0, 0]	Tenseur 3D
arr[-1, -1]	Dernier élément en 2D

— SLICING —

arr[1:5]	Éléments 1 à 4
arr[:, 2]	Un élément sur deux
arr[::-1]	Tableau inversé
arr[1:7:2]	Éléments 1, 3, 5
arr[1:5, 2:6]	Sous-matrice (vue)
arr[:, 0]	Première colonne (vecteur)
arr[0, :]	Première ligne (vecteur)
arr[0:3, 0:3]	Sous-matrice 3×3
arr[..., 0]	Première valeur de la dernière dim
arr[0, ..., -1]	Ellipsis pour dimensions intermédiaires
arr[:, :, np.newaxis]	Ajoute dimension avec slicing

— INDEXATION AVANCÉE (FANCY INDEXING) —

arr[[0, 2, 4]]	Éléments aux indices 0, 2, 4
arr[[0, 2], [1, 3]]	Éléments (0,1) et (2,3)
arr[[0, 1, 2], :]	Lignes 0, 1, 2
arr[:, [0, 2]]	Colonnes 0 et 2
arr[np.array([0, 2, 4])]	Fancy indexing avec array
# Indexation booléenne	
arr[arr > 5]	Éléments > 5 (retourne un vecteur)
arr[arr > 5] = 0	Met à zéro les éléments > 5
mask = (arr > 2) & (arr < 8)	Masque combiné
arr[mask]	Éléments dans l'intervalle
arr[~mask]	Éléments hors de l'intervalle
np.where(arr > 0, arr, 0)	Remplace négatives par 0
np.where(arr > 0, arr, -arr)	Valeur absolue via where
np.nonzero(arr)	Indices des éléments non-nuls
arr[np.nonzero(arr)]	Éléments non-nuls
np.argwhere(arr > 5)	Indices (array 2D) des vrais
np.flatnonzero(arr)	Indices aplatis des non-nuls

— SÉLECTION ET EXTRACTION —

np.take(arr, [0, 2, 4])	Prend des éléments par indices
np.take(arr, [0, 2], axis=0)	Sur un axe spécifique
np.take_along_axis(arr, indices, axis=0)	Indices le long d'un axe
np.compress(condition, arr)	Compression par masque booléen
np.compress([1,0,1], arr, axis=0)	Sélectionne des lignes
np.choose(indices, [a0, a1, a2])	Sélectionne depuis plusieurs arrays
np.select([c1, c2, c3], [a1, a2, a3], default=0)	Sélection conditionnelle multiple
np.extract(condition, arr)	Équivalent de arr[condition]
np.diag(arr)	Diagonale principale
np.diagonal(arr, offset=1)	Diagonale avec décalage
np.trace(arr)	Somme de la diagonale

— VUES ET COPIES —

arr[:]	Vue (pas de copie)
arr.copy()	Copie complète
arr.view()	Vue avec même données
np.shares_memory(arr1, arr2)	Vrai si partagent la mémoire
np.may_share_memory(arr1, arr2)	Version prudente

EXEMPLES :

```
# Top-k indices
top5 = np.argsort(arr)[-5:][::-1]
# Extraction diagonale → vecteur
diag = arr[np.arange(n), np.arange(n)]
```

```
# Masque sur matrice
arr[arr < 0] = 0
```

```
# ReLU
```

[arr] Manipulation de tableaux – concatenate, stack, split, tile, repeat
--

DESCRIPTION : Combine, divise et répète des tableaux
 POURQUOI : Construire des datasets, combiner des résultats, créer des batches.
 CONTEXTE : Preprocessing, augmentation de données, assemblage.

— CONCATÉNATION ET EMPILEMENT —

np.concatenate([a, b])	Concatène sur l'axe 0 (défaut)
np.concatenate([a, b], axis=1)	Concatène sur l'axe 1
np.concatenate([a, b], axis=None)	Aplatit puis concatène
np.vstack([a, b])	Empile verticalement (axe 0)
np.vstack([a, b, c])	Plusieurs tableaux
np.hstack([a, b])	Empile horizontalement (axe 1)
np.dstack([a, b])	Empile en profondeur (axe 2)
np.column_stack([a, b])	Empile comme colonnes (1D→2D)
np.row_stack([a, b])	Identique à vstack
np.stack([a, b])	Crée un nouvel axe (axe 0)
np.stack([a, b], axis=0)	Nouvel axe en 0
np.stack([a, b], axis=1)	Nouvel axe en 1
np.stack([a, b], axis=-1)	Nouvel axe en dernier
np.block([a, b], [c, d])	Assemblage matriciel par blocs

— DIVISION —

np.split(arr, 3)	Divise en 3 parties égales
np.split(arr, [3, 7])	Divise aux indices 3 et 7
np.split(arr, 3, axis=1)	Divise sur l'axe 1
np.vsplit(arr, 3)	Divise verticalement (axe 0)
np.hsplit(arr, 3)	Divise horizontalement (axe 1)
np.dsplit(arr, 3)	Divise en profondeur (axe 2)
np.array_split(arr, 3)	Divise même inégalement (pas d'erreur)
np.array_split(arr, [3, 7])	Divise aux indices 3 et 7

— RÉPÉTITION ET TUILE —

np.tile(arr, 3)	Répète 3 fois (vecteur)
np.tile(arr, (2, 3))	Répète 2× verticalement, 3× horizontalement
np.tile(arr, (1, 1, 3))	Ajoute une dimension et répète
np.repeat(arr, 3)	Répète chaque élément 3 fois
np.repeat(arr, 3, axis=0)	Répète chaque ligne 3 fois
np.repeat(arr, [1,2,3], axis=0)	Répétitions différentes par ligne
arr.repeat(3)	Méthode équivalente

— ROTATION ET FLIP —

np.flip(arr)	Inverse tous les axes
np.flip(arr, axis=0)	Inverse l'axe 0
np.flip(arr, axis=(0, 1))	Inverse plusieurs axes
np.fliplr(arr)	Inverse gauche-droite (axe 1)
np.flipud(arr)	Inverse haut-bas (axe 0)
np.rot90(arr)	Rotation 90° antihoraire
np.rot90(arr, k=2)	Rotation 180°
np.rot90(arr, k=-1)	Rotation 90° horaire
np.rot90(arr, axes=(1, 2))	Rotation dans un plan 3D
np.roll(arr, 2)	Décale les éléments de 2 positions
np.roll(arr, 2, axis=0)	Décale les lignes
np.roll(arr, -1, axis=1)	Décale les colonnes vers la gauche

— REMBOURRAGE —

np.pad(arr, 1)	Rembourrage de 1 de chaque côté
np.pad(arr, (2, 3))	2 avant, 3 après
np.pad(arr, ((1,2),(3,4)))	Rembourrage par axe
np.pad(arr, 1, mode="constant")	Avec une constante (0 par défaut)
np.pad(arr, 1, constant_values=0)	Valeur constante
np.pad(arr, 1, mode="edge")	Reproduit les bords
np.pad(arr, 1, mode="reflect")	Réflexion symétrique
np.pad(arr, 1, mode="wrap")	Rembourrage cyclique
np.pad(arr, 1, mode="linear_ramp")	Rampe linéaire vers 0

— INSERTION ET SUPPRESSION —

np.insert(arr, 2, 99)	Insère 99 à l'indice 2
np.insert(arr, [2,4], [99,100])	Insère plusieurs valeurs
np.insert(arr, 2, 99, axis=0)	Insère une ligne
np.delete(arr, 2)	Supprime l'élément à l'indice 2

<code>np.delete(arr, [0, 2])</code>	Supprime plusieurs éléments
<code>np.delete(arr, 2, axis=0)</code>	Supprime une ligne
<code>np.append(arr, [7, 8])</code>	Ajoute à la fin (retourne copie)
<code>np.append(arr, [[7],[8]], axis=1)</code>	Ajoute une colonne

EXEMPLES :

```
# Assembler des batches
X = np.stack([img1, img2, img3], axis=0) # (3, H, W)
# Découper en mini-batches de 32
batches = np.array_split(X, len(X)//32)
# Rembourrage pour convolution
X_padded = np.pad(X, ((0,0),(1,1),(1,1)), mode="constant")
```

[np] Opérations arithmétiques et mathématiques

DESCRIPTION : Opérations mathématiques élément par élément et réductions
 POURQUOI : Calculs numériques vectorisés, rapides et concis.
 CONTEXTE : Tout calcul scientifique, ML, finance, signal.

— OPÉRATEURS ARITHMÉTIQUES (ELEMENT-WISE) —

<code>a + b</code>	Addition
<code>a - b</code>	Soustraction
<code>a * b</code>	Multiplication élément par élément
<code>a / b</code>	Division réelle
<code>a // b</code>	Division entière
<code>a % b</code>	Modulo
<code>a ** b</code>	Puissance
<code>-a</code>	Négation
<code>np.add(a, b)</code>	Addition (ufunc)
<code>np.subtract(a, b)</code>	Soustraction (ufunc)
<code>np.multiply(a, b)</code>	Multiplication (ufunc)
<code>np.divide(a, b)</code>	Division réelle (ufunc)
<code>np.floor_divide(a, b)</code>	Division entière (ufunc)
<code>np.mod(a, b)</code>	Modulo (ufunc)
<code>np.power(a, b)</code>	Puissance (ufunc)
<code>np.negative(a)</code>	Négation (ufunc)
<code>np.positive(a)</code>	Identité (ufunc)
<code>np.reciprocal(a)</code>	1/a
<code>np.absolute(a)</code>	Valeur absolue
<code>np.abs(a)</code>	Alias
<code>np.fabs(a)</code>	Valeur absolue (float seulement)
<code>np.sign(a)</code>	Signe (-1, 0, +1)
<code>np.copysign(a, b)</code>	Signe de b, magnitude de a
<code># Opérations in-place</code>	
<code>np.add(a, b, out=a)</code>	Stocke dans a (sans copie)
<code>np.multiply(a, b, out=c)</code>	Stocke dans c

— FONCTIONS MATHÉMATIQUES —

<code>np.sqrt(arr)</code>	Racine carrée
<code>np.cbrt(arr)</code>	Racine cubique
<code>np.square(arr)</code>	Carré (arr^2)
<code>np.exp(arr)</code>	Exponentielle e^x
<code>np.exp2(arr)</code>	2^x
<code>np.expm1(arr)</code>	$e^x - 1$ (précis pour $x \approx 0$)
<code>np.log(arr)</code>	Logarithme naturel (ln)
<code>np.log2(arr)</code>	Logarithme base 2
<code>np.log10(arr)</code>	Logarithme base 10
<code>np.log1p(arr)</code>	$\ln(1+x)$ (précis pour $x \approx 0$)
<code>np.logaddexp(a, b)</code>	$\log(e^a + e^b)$ (stabilité numérique)
<code>np.logaddexp2(a, b)</code>	$\log(2^a + 2^b)$

— TRIGONOMÉTRIE —

<code>np.sin(arr)</code>	Sinus (radians)
<code>np.cos(arr)</code>	Cosinus (radians)
<code>np.tan(arr)</code>	Tangente (radians)
<code>np.arcsin(arr)</code>	Arc sinus
<code>np.arccos(arr)</code>	Arc cosinus
<code>np.arctan(arr)</code>	Arc tangente
<code>np.arctan2(y, x)</code>	Arc tangente de y/x (4 quadrants)
<code>np.hypot(a, b)</code>	$\sqrt{a^2 + b^2}$
<code>np.degrees(arr)</code>	Radians → degrés
<code>np.radians(arr)</code>	Degrés → radians
<code>np.deg2rad(arr)</code>	Alias de radians
<code>np.rad2deg(arr)</code>	Alias de degrees

np.unwrap(arr)	Dépliage de phase
# Hyperboliques	
np.sinh(arr)	Sinus hyperbolique
np.cosh(arr)	Cosinus hyperbolique
np.tanh(arr)	Tangente hyperbolique
np.arcsinh(arr)	Arc sinus hyperbolique
np.arccosh(arr)	Arc cosinus hyperbolique
np.arctanh(arr)	Arc tangente hyperbolique
— ARRONDI —	
np.round(arr, 2)	Arrondi à 2 décimales
np.round_(arr, 2)	Arrondi in-place
arr.round(2)	Méthode équivalente
np.floor(arr)	Arrondi inférieur ($\lfloor x \rfloor$)
np.ceil(arr)	Arrondi supérieur ($\lceil x \rceil$)
np.trunc(arr)	Troncature vers zéro
np.fix(arr)	Troncature vers zéro (alias)
np rint(arr)	Arrondi à l'entier le plus proche
np.around(arr, 3)	Alias de round
np.spacing(arr)	Plus petit incrément depuis arr
— OPÉRATIONS BIT À BIT —	
np.bitwise_and(a, b)	ET bit à bit
np.bitwise_or(a, b)	OU bit à bit
np.bitwise_xor(a, b)	XOR bit à bit
np.bitwise_not(a)	NON bit à bit
np.invert(a)	Alias de bitwise_not
np.left_shift(a, n)	Décalage à gauche de n bits
np.right_shift(a, n)	Décalage à droite de n bits
a & b	ET bit à bit (opérateur)
a b	OU bit à bit (opérateur)
a ^ b	XOR bit à bit (opérateur)
~a	NON bit à bit (opérateur)
— OPÉRATIONS LOGIQUES —	
np.logical_and(a, b)	ET logique élément par élément
np.logical_or(a, b)	OU logique
np.logical_xor(a, b)	XOR logique
np.logical_not(a)	NON logique
np.all(arr)	Vrai si tous les éléments vrais
np.all(arr, axis=0)	Sur l'axe 0
np.any(arr)	Vrai si au moins un élément vrai
np.any(arr, axis=1)	Sur l'axe 1
— COMPARAISONS —	
np.equal(a, b)	a == b (ufunc)
np.not_equal(a, b)	a != b (ufunc)
np.less(a, b)	a < b (ufunc)
np.less_equal(a, b)	a <= b (ufunc)
np.greater(a, b)	a > b (ufunc)
np.greater_equal(a, b)	a >= b (ufunc)
np.array_equal(a, b)	Vrai si même shape et valeurs
np.array_equiv(a, b)	Vrai si équivalents via broadcast
np.allclose(a, b)	Vrai si tous les éléments proches
np.allclose(a, b, rtol=1e-5, atol=1e-8)	Avec tolérances
np.isclose(a, b)	Tableau booléen élément par élément
np.isclose(a, b, rtol=1e-5, equal_nan=True)	Considère NaN comme égaux
— FONCTIONS SPÉCIALES —	
np.clip(arr, a_min, a_max)	Borne les valeurs
np.clip(arr, 0, None)	Borne seulement par le bas
arr.clip(0, 1)	Méthode équivalente
np.where(condition, x, y)	Ternaire élément par élément
np.where(condition)	Indices des éléments vrais
np.piecewise(arr, [c1,c2], [f1,f2])	Fonctions par morceaux
np.interp(x, xp, fp)	Interpolation linéaire 1D
np.interp(x, xp, fp, left=0, right=1)	Avec valeurs aux bords
np.gradient(arr)	Gradient numérique
np.gradient(arr, dx)	Avec pas de discrétisation
np.gradient(arr, axis=0)	Sur l'axe 0
np.diff(arr)	Différences consécutives
np.diff(arr, n=2)	Différences secondes
np.diff(arr, axis=0)	Sur l'axe 0
np.ediff1d(arr)	Différences 1D
np.cumsum(arr)	Somme cumulée

np.cumsum(arr, axis=0)	Sur l'axe 0
np.cumprod(arr)	Produit cumulé
np.nancumsum(arr)	Somme cumulée (ignore NaN)
np.nancumprod(arr)	Produit cumulé (ignore NaN)
np.convolve(a, b)	Convolution 1D
np.convolve(a, b, mode="full")	Mode : full, valid, same
np.correlate(a, b)	Corrélation croisée 1D
np.cross(a, b)	Produit vectoriel

EXEMPLES :

```
# Fonction sigmoïde
def sigmoid(x): return 1 / (1 + np.exp(-x))
# Normalisation
arr_norm = (arr - arr.min()) / (arr.max() - arr.min())
# Softmax
def softmax(x): e = np.exp(x - x.max()); return e / e.sum()
```

[np] Statistiques et réductions

DESCRIPTION : Calcule des statistiques descriptives sur les tableaux
 POURQUOI : Analyser et résumer les données numériques.
 CONTEXTE : Analyse exploratoire, normalisation, évaluation de modèles.

RÉDUCTIONS PRINCIPALES :

np.sum(arr)	Somme de tous les éléments
np.sum(arr, axis=0)	Somme par colonne
np.sum(arr, axis=1)	Somme par ligne
np.sum(arr, axis=(0,1))	Somme sur plusieurs axes
np.sum(arr, keepdims=True)	Conserve les dimensions
arr.sum()	Méthode équivalente
np.prod(arr)	Produit de tous les éléments
np.prod(arr, axis=0)	Produit par colonne
np.mean(arr)	Moyenne
np.mean(arr, axis=0)	Moyenne par colonne
np.mean(arr, axis=1, keepdims=True)	Moyenne par ligne (avec dim.)
arr.mean()	Méthode équivalente
np.median(arr)	Médiane
np.median(arr, axis=0)	Médiane par colonne
np.percentile(arr, 50)	Percentile 50 (= médiane)
np.percentile(arr, [25, 50, 75])	Quartiles
np.percentile(arr, 95, axis=0)	Sur l'axe 0
np.percentile(arr, 50, interpolation="linear")	Méthode d'interpolation
np.quantile(arr, 0.5)	Quantile (0-1 au lieu de 0-100)
np.quantile(arr, [0.25, 0.75])	Quartiles (0-1)
np.std(arr)	Écart-type (population, ddof=0)
np.std(arr, ddof=1)	Écart-type échantillon
np.std(arr, axis=0)	Par colonne
arr.std()	Méthode équivalente
np.var(arr)	Variance
np.var(arr, ddof=1)	Variance échantillon
np.min(arr)	Minimum
np.min(arr, axis=0)	Minimum par colonne
arr.min()	Méthode équivalente
np.max(arr)	Maximum
np.max(arr, axis=0)	Maximum par colonne
np.ptp(arr)	Étendue (max - min)
np.ptp(arr, axis=0)	Étendue par colonne
np.argmin(arr)	Indice du minimum
np.argmin(arr, axis=0)	Indice du min par colonne
arr.argmin()	Méthode équivalente
np.argmax(arr)	Indice du maximum
np.argmax(arr, axis=0)	Indice du max par colonne
np.nansum(arr)	Somme (ignore NaN)
np.nanmean(arr)	Moyenne (ignore NaN)
np.nanmedian(arr)	Médiane (ignore NaN)
np.nanstd(arr)	Écart-type (ignore NaN)
np.nanvar(arr)	Variance (ignore NaN)
np.nanmin(arr)	Minimum (ignore NaN)
np.nanmax(arr)	Maximum (ignore NaN)
np.nanargmin(arr)	Indice du min (ignore NaN)
np.nanargmax(arr)	Indice du max (ignore NaN)
np.nanpercentile(arr, 50)	Percentile (ignore NaN)
np.nanquantile(arr, 0.5)	Quantile (ignore NaN)

CORRÉLATIONS ET COVARIANCES :

<code>np.corrcoef(x, y)</code>	Matrice de corrélation de Pearson
<code>np.corrcoef(matrice)</code>	Corrélation des lignes
<code>np.corrcoef(matrice, rowvar=False)</code>	Corrélation des colonnes
<code>np.cov(x, y)</code>	Matrice de covariance
<code>np.cov(matrice, rowvar=True)</code>	Covariance des lignes (défaut)
<code>np.cov(matrice, rowvar=False)</code>	Covariance des colonnes
<code>np.cov(x, aweights=poids)</code>	Covariance pondérée
<code>np.histogram(arr, bins=10)</code>	Histogramme 1D → (counts, edges)
<code>np.histogram(arr, bins=[0,5,10,20])</code>	Avec bins personnalisés
<code>np.histogram(arr, density=True)</code>	Densité normalisée
<code>np.histogram2d(x, y, bins=10)</code>	Histogramme 2D
<code>np.histogramdd(données, bins=5)</code>	Histogramme N-D
<code>np.bincount(arr)</code>	Compte les entiers (arr doit être int)
<code>np.bincount(arr, weights=w)</code>	Compte pondéré
<code>np.digitize(arr, bins)</code>	Indices des classes pour chaque élément
<code>np.digitize(arr, bins, right=True)</code>	Bords à droite

EXEMPLES :

```
# Z-score normalisation
z = (arr - np.mean(arr)) / np.std(arr)
# Statistiques par groupe (broadcasting)
moyennes = arr.mean(axis=1, keepdims=True)
arr_centré = arr - moyennes
```

[np] Tri, recherche et valeurs uniques
--

DESCRIPTION : Trie, cherche et identifie les valeurs uniques
POURQUOI : Ordonner les données, trouver des éléments, dédupliquer.
CONTEXTE : Classement, recherche, préparation de données.

TRI :

<code>np.sort(arr)</code>	Retourne un tableau trié (copie)
<code>np.sort(arr, axis=0)</code>	Tri sur l'axe 0 (par colonne)
<code>np.sort(arr, axis=1)</code>	Tri sur l'axe 1 (par ligne)
<code>np.sort(arr, kind="quicksort")</code>	Algorithme : quicksort (défaut)
<code>np.sort(arr, kind="mergesort")</code>	Tri stable (mergesort)
<code>np.sort(arr, kind="heapsort")</code>	Heapsort
<code>np.sort(arr, kind="stable")</code>	Tri stable (mergesort/timsort)
<code>arr.sort()</code>	Tri in-place
<code>arr.sort(axis=0)</code>	Tri in-place sur l'axe 0
<code>np.argsort(arr)</code>	Indices qui trieraient arr
<code>np.argsort(arr, axis=0)</code>	Indices par colonne
<code>np.argsort(arr)[::-1]</code>	Indices du plus grand au plus petit
<code>np.argsort(arr, kind="stable")</code>	Tri stable
<code>np.lexsort((clé2, clé1))</code>	Tri multi-clés (clé1 primaire)
<code>np.msort(arr)</code>	Tri sur l'axe 0 (déprécié)
<code>np.partition(arr, kth=3)</code>	Partition (élément kth à sa place)
<code>np.partition(arr, kth=-5)</code>	5 plus grands à la fin
<code>np.argpartition(arr, kth=3)</code>	Indices de la partition

RECHERCHE :

<code>np.searchsorted(arr_trié, valeur)</code>	Indice d'insertion (bin. search)
<code>np.searchsorted(arr_trié, valeur, side="left")</code>	Côté gauche (défaut)
<code>np.searchsorted(arr_trié, valeur, side="right")</code>	Côté droit
<code>np.searchsorted(arr_trié, [v1,v2])</code>	Plusieurs valeurs
<code>np.where(arr == valeur)</code>	Indices des occurrences
<code>np.argwhere(arr == valeur)</code>	Indices en format 2D
<code>np.nonzero(arr)</code>	Indices des non-nuls

VALEURS UNIQUES :

<code>np.unique(arr)</code>	Valeurs uniques (triées)
<code>np.unique(arr, return_index=True)</code>	+ indices première occurrence
<code>np.unique(arr, return_inverse=True)</code>	+ indices de reconstruction
<code>np.unique(arr, return_counts=True)</code>	+ comptage de chaque valeur
<code>np.unique(arr, axis=0)</code>	Lignes uniques
<code>np.in1d(arr, valeurs)</code>	Masque booléen d'appartenance
<code>np.in1d(arr, valeurs, invert=True)</code>	Négation
<code>np.isin(arr, valeurs)</code>	Identique à in1d (N-D)
<code>np.isin(arr, valeurs, invert=True)</code>	Négation
<code>np.intersect1d(a, b)</code>	Intersection
<code>np.intersect1d(a, b, assume_unique=True)</code>	Plus rapide si déjà uniques
<code>np.union1d(a, b)</code>	Union
<code>np.setdiff1d(a, b)</code>	Différence a \ b

np.setxor1d(a, b) Différence symétrique

```
EXEMPLES :
# Top-k éléments
top_k = arr[np.argpartition(arr, -k)[-k:]]
# Rang de chaque élément
rangs = np.argsort(np.argsort(arr))
# Encodage label
uniques, indices = np.unique(labels, return_inverse=True)
```

[np] Broadcasting et vectorisation

DESCRIPTION : Applique des opérations à des tableaux de formes différentes
POURQUOI : Éviter les boucles Python, performances maximales.
CONTEXTE : Normalisation, distances, opérations batch.

RÈGLES DU BROADCASTING :

```
# Les formes sont alignées depuis la droite
# Une dimension de taille 1 est étirée pour correspondre
(3, 4) + (4,) → (3, 4) # Vecteur ajouté à chaque ligne
(3, 1) + (1, 4) → (3, 4) # Produit extérieur
(3,) + (3, 1) → (3, 3) # Vecteur ligne × vecteur colonne
(1, 3, 4) * (2, 1, 4) → (2, 3, 4) # Tenseurs
```

FONCTIONS DE BROADCASTING :

```
np.broadcast_to(arr, shape) Étire arr vers shape (vue)
np.broadcast_shapes(s1, s2) Calcule la shape résultante
np.broadcast_arrays(a, b) Étire a et b pour correspondre
np.broadcast(a, b) Objet d'itération broadcast
```

VECTORISATION :

```
np.vectorize(func) Vectorise une fonction Python
np.vectorize(func, otypes=[float]) Avec type de retour explicite
np.vectorize(func, excluded=[0]) Argument exclu du vectorize
vfunc = np.vectorize(func)
vfunc(arr1, arr2) Applique func élément par élément
np.frompyfunc(func, nin, nout) Crée une ufunc depuis une fonction
# Produit extérieur
np.outer(a, b) Produit extérieur a ⊗ b
np.outer(a, b) == a[:, None] * b[None, :] Équivalent par broadcasting
np.kron(a, b) Produit de Kronecker
np.einsum("ij,jk->ik", A, B) Notation einsum (produit matriciel)
np.einsum("ii->i", A) Extraction diagonale
np.einsum("ij->i", A) Somme par ligne
np.einsum("ij->", A) Somme totale
np.einsum("ij,ij->", A, B) Produit scalaire de matrices aplaties
np.einsum("ijk,ikl->ijl", A, B) Batch matrix multiply
np.einsum("ij,ij->i", A, B) Norme au carré par ligne
np.einsum_path("ij,jk,kl->il", A,B,C) Chemin d'optimisation einsum
```

EXEMPLES :

```
# Normalisation de chaque ligne
arr_norm = arr / arr.sum(axis=1, keepdims=True)
# Distance euclidienne entre tous les points
diff = X[:, None, :] - X[None, :, :] # (N, N, D)
dist = np.sqrt((diff**2).sum(axis=-1)) # (N, N)
# Softmax stable
e = np.exp(arr - arr.max(axis=-1, keepdims=True))
softmax = e / e.sum(axis=-1, keepdims=True)
```

[np] Algèbre linéaire – np.linalg

DESCRIPTION : Opérations d'algèbre linéaire sur matrices et vecteurs
POURQUOI : Résolution de systèmes, décompositions, calcul de normes.
CONTEXTE : Machine learning, optimisation, traitement du signal, statistiques.

PRODUITS MATRICIELS :

```
np.dot(a, b) Produit scalaire / matriciel
a @ b Opérateur de produit matriciel
np.matmul(a, b) Produit matriciel (pas de scalaire)
np.inner(a, b) Produit intérieur
np.outer(a, b) Produit extérieur
np.tensordot(a, b, axes=1) Produit tensoriel
```

<code>np.tensordot(a, b, axes=([1],[0]))</code>	Axes spécifiés
<code>np.einsum("ij,jk->ik", a, b)</code>	Produit matriciel via einsum
<code>np.cross(a, b)</code>	Produit vectoriel
DÉCOMPOSITIONS :	
<code>np.linalg.eig(A)</code>	Valeurs et vecteurs propres
<code>np.linalg.eig(A)</code>	→ (valeurs, vecteurs)
<code>np.linalg.eigh(A)</code>	Matrices hermitiennes / symétriques
<code>np.linalg.eigvals(A)</code>	Valeurs propres seulement
<code>np.linalg.eigvalsh(A)</code>	Valeurs propres (symétrique)
<code>np.linalg.svd(A)</code>	Décomposition SVD
<code>np.linalg.svd(A)</code>	→ (U, S, Vt)
<code>np.linalg.svd(A, full_matrices=False)</code>	SVD compacte (économique)
<code>np.linalg.svd(A, compute_uv=False)</code>	Valeurs singulières seulement
<code>np.linalg.qr(A)</code>	Décomposition QR → (Q, R)
<code>np.linalg.qr(A, mode="complete")</code>	QR complète
<code>np.linalg.qr(A, mode="reduced")</code>	QR réduite (défaut)
<code>np.linalg.cholesky(A)</code>	Décomposition de Cholesky → L
<code>np.linalg.lu_factor(A)</code>	Factorisation LU (via scipy)
RÉSOLUTION DE SYSTÈMES :	
<code>np.linalg.solve(A, b)</code>	Résout $Ax = b$
<code>np.linalg.lstsq(A, b)</code>	Moindres carrés $Ax \approx b$
<code>np.linalg.lstsq(A, b, rcond=None)</code>	Avec seuil de coupure
<code>np.linalg.lstsq(A, b)</code>	→ (x, résidus, rang, valeurs_s)
<code>np.linalg.inv(A)</code>	Matrice inverse
<code>np.linalg.pinv(A)</code>	Pseudo-inverse (Moore-Penrose)
<code>np.linalg.pinv(A, rcond=1e-15)</code>	Avec seuil de coupure
NORMES ET PROPRIÉTÉS :	
<code>np.linalg.norm(v)</code>	Norme L2 d'un vecteur
<code>np.linalg.norm(v, ord=1)</code>	Norme L1
<code>np.linalg.norm(v, ord=np.inf)</code>	Norme L^∞
<code>np.linalg.norm(A)</code>	Norme de Frobenius d'une matrice
<code>np.linalg.norm(A, ord=2)</code>	Norme spectrale (max valeur sing.)
<code>np.linalg.norm(A, ord="fro")</code>	Norme de Frobenius explicite
<code>np.linalg.norm(A, axis=0)</code>	Norme de chaque colonne
<code>np.linalg.norm(A, axis=1)</code>	Norme de chaque ligne
<code>np.linalg.det(A)</code>	Déterminant
<code>np.linalg.slogdet(A)</code>	Signe et log du déterminant
<code>np.linalg.matrix_rank(A)</code>	Rang de la matrice
<code>np.linalg.matrix_rank(A, tol=1e-10)</code>	Avec tolérance
<code>np.linalg.cond(A)</code>	Nombre de conditionnement
<code>np.linalg.cond(A, p=2)</code>	Norm L2
<code>np.linalg.trace(A)</code>	Trace (identique à <code>np.trace</code>)
<code>np.linalg.matrix_power(A, n)</code>	Puissance matricielle A^n
<code>np.linalg.multi_dot([A, B, C, D])</code>	Produit multiple optimisé
EXEMPLES :	
# Régression linéaire (moindres carrés)	
<code>coeffs, _, _, _ = np.linalg.lstsq(X_aug, y, rcond=None)</code>	
# PCA manuelle	
<code>U, S, Vt = np.linalg.svd(X - X.mean(axis=0), full_matrices=False)</code>	
<code>composantes = Vt[:k]</code>	
# Résolution système linéaire	
<code>x = np.linalg.solve(A, b)</code>	

[np] Nombres aléatoires – `np.random / default_rng`

DESCRIPTION : Génère des nombres et tableaux aléatoires
POURQUOI : Simulations, initialisation de modèles, bootstrap, tests.
CONTEXTE : Machine learning, statistiques, simulations Monte-Carlo.

— API MODERNE (RECOMMANDÉE depuis NumPy 1.17)

<code>rng = np.random.default_rng()</code>	Générateur par défaut
<code>rng = np.random.default_rng(seed=42)</code>	Avec graine reproductible
<code>rng = np.random.default_rng(seed=None)</code>	Graine aléatoire (défaut)
# Distributions uniformes	
<code>rng.random()</code>	Flottant uniforme [0, 1)
<code>rng.random(size=100)</code>	100 flottants [0, 1)
<code>rng.random(size=(3, 4))</code>	Matrice 3×4
<code>rng.integers(0, 10)</code>	Entier uniforme [0, 10)
<code>rng.integers(0, 10, size=100)</code>	100 entiers
<code>rng.integers(0, 10, size=(3,4))</code>	Matrice d'entiers

<code>rng.integers(0, 10, endpoint=True)</code>	<code>[0, 10]</code> inclus
<code>rng.uniform(low=0, high=1)</code>	Uniforme <code>[0, 1)</code>
<code>rng.uniform(low=-1, high=1, size=(100,))</code>	Vecteur uniforme <code>[-1, 1)</code>
# Distributions gaussiennes	
<code>rng.normal()</code>	<code>N(0, 1)</code>
<code>rng.normal(loc=0, scale=1, size=100)</code>	<code>N(μ, σ)</code> avec 100 échantillons
<code>rng.normal(loc=5, scale=2, size=(3,4))</code>	Matrice 3x4
<code>rng.standard_normal(size=100)</code>	<code>N(0, 1)</code> standard
<code>rng.multivariate_normal(mean, cov, size=100)</code>	Distribution multivariée
# Autres distributions	
<code>rng.exponential(scale=1, size=100)</code>	Distribution exponentielle
<code>rng.poisson(lam=5, size=100)</code>	Distribution de Poisson
<code>rng.binomial(n=10, p=0.5, size=100)</code>	Distribution binomiale
<code>rng.multinomial(n=10, pvals=[.2,.3,.5])</code>	Distribution multinomiale
<code>rng.geometric(p=0.5, size=100)</code>	Distribution géométrique
<code>rng.hypergeometric(ngood=10, nbad=5, nsample=8)</code>	Distribution hypergéométrique
<code>rng.beta(a=2, b=5, size=100)</code>	Distribution beta
<code>rng.gamma(shape=2, scale=1, size=100)</code>	Distribution gamma
<code>rng.chi_square(df=5, size=100)</code>	Distribution chi-deux
<code>rng.f(dfnum=5, dfden=2, size=100)</code>	Distribution F
<code>rng.lognormal(mean=0, sigma=1, size=100)</code>	Distribution log-normale
<code>rng.logistic(loc=0, scale=1, size=100)</code>	Distribution logistique
<code>rng.laplace(loc=0, scale=1, size=100)</code>	Distribution de Laplace
<code>rng.rayleigh(scale=1, size=100)</code>	Distribution de Rayleigh
<code>rng.vonmises(mu=0, kappa=1, size=100)</code>	Distribution de Von Mises
<code>rng.wald(mean=1, scale=1, size=100)</code>	Distribution de Wald
<code>rng.weibull(a=1.5, size=100)</code>	Distribution de Weibull
<code>rng.dirichlet(alpha=[1,2,3], size=100)</code>	Distribution de Dirichlet
<code>rng.standard_cauchy(size=100)</code>	Distribution de Cauchy
<code>rng.standard_t(df=5, size=100)</code>	Distribution de Student
# Permutations et tirages	
<code>rng.shuffle(arr)</code>	Mélange in-place
<code>rng.permutation(arr)</code>	Retourne une permutation (copie)
<code>rng.permutation(10)</code>	Permutation de <code>np.arange(10)</code>
<code>rng.choice(arr, size=10)</code>	Tirage avec remise
<code>rng.choice(arr, size=10, replace=False)</code>	Tirage sans remise
<code>rng.choice(arr, size=10, p=proba)</code>	Tirage pondéré
<code>rng.choice(5, size=10)</code>	Tirage depuis <code>range(5)</code>
<code>rng.bytes(10)</code>	10 bytes aléatoires

— ANCIENNE API (`np.random.*`) —

<code>np.random.seed(42)</code>	Graine globale
<code>np.random.rand(3, 4)</code>	Uniforme <code>[0, 1)</code> , shape (3,4)
<code>np.random.randn(3, 4)</code>	Normal standard <code>N(0,1)</code> , shape (3,4)
<code>np.random.randint(0, 10, size=100)</code>	Entiers <code>[0, 10)</code>
<code>np.random.random(size=100)</code>	Uniforme <code>[0, 1)</code>
<code>np.random.normal(0, 1, size=100)</code>	Normal
<code>np.random.uniform(0, 1, size=100)</code>	Uniforme <code>[0, 1)</code>
<code>np.random.choice(arr, size=10)</code>	Tirage
<code>np.random.shuffle(arr)</code>	Mélange in-place
<code>np.random.permutation(arr)</code>	Permutation (copie)

EXEMPLES :

```

rng = np.random.default_rng(seed=42)
# Monte-Carlo  $\pi$ 
x, y = rng.random(1_000_000), rng.random(1_000_000)
pi_approx = 4 * np.mean(x**2 + y**2 < 1)
# Initialisation Xavier
W = rng.normal(0, np.sqrt(2/n_in), size=(n_in, n_out))
# Bootstrap
indices = rng.integers(0, len(X), size=(B, len(X)))
bootstrap_means = X[indices].mean(axis=1)

```

[np] Transformées de Fourier — `np.fft`

DESCRIPTION : Transformées de Fourier discrètes
POURQUOI : Analyse fréquentielle de signaux, filtrage, convolution rapide.
CONTEXTE : Traitement du signal, audio, images, séries temporelles.

FONCTIONS PRINCIPALES :

<code>np.fft.fft(arr)</code>	DFT 1D complexe
<code>np.fft.fft(arr, n=512)</code>	Avec zero-padding à n points
<code>np.fft.fft(arr, axis=0)</code>	Sur l'axe 0
<code>np.fft.ifft(spectre)</code>	DFT inverse 1D

<code>np.fft.rfft(arr)</code>	DFT réelle 1D (spectre positif)
<code>np.fft.irfft(spectre)</code>	DFT réelle inverse
<code>np.fft.irfft(spectre, n=len(arr))</code>	Avec taille de sortie explicite
<code>np.fft.fft2(image)</code>	DFT 2D
<code>np.fft.ifft2(spectre)</code>	DFT 2D inverse
<code>np.fft.rfft2(image)</code>	DFT 2D réelle
<code>np.fft.irfft2(spectre)</code>	DFT 2D réelle inverse
<code>np.fft.fftn(arr)</code>	DFT N-dimensionnelle
<code>np.fft.ifftn(spectre)</code>	DFT N-D inverse
<code>np.fft.rfftn(arr)</code>	DFT N-D réelle
<code>np.fft.irfftn(spectre)</code>	DFT N-D réelle inverse
<code>np.fft.fftshift(spectre)</code>	Déplace la fréquence 0 au centre
<code>np.fft.ifftshift(spectre)</code>	Inverse du <code>fftshift</code>
<code>np.fft.fftfreq(n, d=1)</code>	Fréquences associées (axe <code>fft</code>)
<code>np.fft.fftfreq(n, d=1/Fs)</code>	Fréquences en Hz (F_s = fréq. d'ech.)
<code>np.fft.rfftfreq(n, d=1)</code>	Fréquences pour <code>rfft</code>

EXEMPLES :

<code>Fs = 1000</code>	# Fréquence d'échantillonnage
<code>spectre = np.fft.rfft(signal)</code>	
<code>freqs = np.fft.rfftfreq(len(signal), d=1/Fs)</code>	
<code>amplitude = np.abs(spectre)</code>	

[np] Gestion des NaN et valeurs spéciales

DESCRIPTION : Détection et traitement des valeurs manquantes et spéciales
 POURQUOI : Les données réelles contiennent souvent des NaN ou des infinis.
 CONTEXTE : Nettoyage de données, calculs robustes.

DÉTECTION :

<code>np.isnan(arr)</code>	Masque des NaN
<code>np.isinf(arr)</code>	Masque des $\pm$ infinis
<code>np.isposinf(arr)</code>	Masque des +infinis
<code>np.isneginf(arr)</code>	Masque des -infinis
<code>np.isfinite(arr)</code>	Masque des valeurs finies
<code>np.isnan(arr).any()</code>	Vrai si au moins un NaN
<code>np.isnan(arr).sum()</code>	Nombre de NaN
<code>np.count_nonzero(np.isnan(arr))</code>	Identique
<code>np.isnan(arr).mean()</code>	Proportion de NaN

TRAITEMENT :

<code>arr[np.isnan(arr)] = 0</code>	Remplace NaN par 0
<code>arr[np.isnan(arr)] = np.nanmean(arr)</code>	Remplace par la moyenne
<code>np.nan_to_num(arr)</code>	NaN→0, +inf→max, -inf→min
<code>np.nan_to_num(arr, nan=0, posinf=999, neginf=-999)</code>	Valeurs custom
<code>np.where(np.isnan(arr), 0, arr)</code>	Remplacement conditionnel
<code>arr_propre = arr[~np.isnan(arr)]</code>	Supprime les NaN

FONCTIONS NAN-SAFE :

<code>np.nansum</code> , <code>np.nanmean</code> , <code>np.nanmedian</code>	(voir section statistiques)
<code>np.nanstd</code> , <code>np.nanvar</code>	(voir section statistiques)
<code>np.nanmin</code> , <code>np.nanmax</code>	(voir section statistiques)
<code>np.nanquantile</code> , <code>np.nanpercentile</code>	(voir section statistiques)
<code>np.nanargmin</code> , <code>np.nanargmax</code>	(voir section statistiques)
<code>np.nancumsum</code> , <code>np.nancumprod</code>	(voir section mathématiques)

[np] Sauvegarde et chargement

DESCRIPTION : Sauvegarde et charge des tableaux numpy dans des fichiers
 POURQUOI : Persister des données, partager des résultats, caching.
 CONTEXTE : Pipelines ML, expériences scientifiques, échange de données.

FORMATS BINAIRES NUMPY :

<code>np.save("fichier.npy", arr)</code>	Sauvegarde un tableau en .npy
<code>arr = np.load("fichier.npy")</code>	Charge un fichier .npy
<code>np.savez("fichier.npz", a=arr1, b=arr2)</code>	Sauvegarde plusieurs tableaux
<code>np.savez("fichier.npz", arr1, arr2)</code>	Sans noms (<code>arr_0</code> , <code>arr_1</code> ...)
<code>np.savez_compressed("fichier.npz", a=arr1)</code>	Compressé (zlib)
<code>data = np.load("fichier.npz")</code>	Charge un .npz → <code>NpzFile</code>
<code>data["a"]</code>	Accède au tableau "a"
<code>data.files</code>	Liste des tableaux dans le .npz
<code>data.close()</code>	Ferme le fichier

FORMATS TEXTE :

np.savetxt("data.txt", arr)	Sauvegarde en texte
np.savetxt("data.csv", arr, delimiter=",")	En CSV
np.savetxt("data.txt", arr, fmt="%.4f")	Format numérique
np.savetxt("data.txt", arr, fmt="%d")	En entiers
np.savetxt("data.txt", arr, header="a b c")	En-tête
np.savetxt("data.txt", arr, comments="# ")	Préfixe des commentaires
arr = np.loadtxt("data.txt")	Charge en texte
arr = np.loadtxt("data.csv", delimiter=",")	Depuis CSV
arr = np.loadtxt("data.txt", skiprows=1)	Saute la première ligne
arr = np.loadtxt("data.txt", usecols=(0,2))	Colonnes spécifiques
arr = np.loadtxt("data.txt", dtype=int)	En entiers
# Fichiers binaires bruts	
arr.tofile("data.bin")	Sauvegarde binaire brute
arr = np.fromfile("data.bin", dtype=float)	Charge binaire brut

EXEMPLES :

```
# Sauvegarder un dataset
np.savez_compressed("dataset.npz", X=X_train, y=y_train)
data = np.load("dataset.npz")
X, y = data["X"], data["y"]
```

[np] Tableaux masqués – numpy.ma

DESCRIPTION : Tableaux avec masque pour ignorer certaines valeurs
 POURQUOI : Traiter des données incomplètes sans modifier les valeurs.
 CONTEXTE : Données manquantes, valeurs aberrantes, mesures invalides.

CRÉATION :

ma.array([1,2,3,4], mask=[0,1,0,0])	Masque manuel (1=masqué)
ma.masked_array(arr, mask=arr<0)	Masque conditionnel
ma.masked_where(arr < 0, arr)	Masque où condition vraie
ma.masked_where(np.isnan(arr), arr)	Masque les NaN
ma.masked_greater(arr, 100)	Masque les valeurs > 100
ma.masked_less(arr, 0)	Masque les valeurs < 0
ma.masked_equal(arr, -9999)	Masque les sentinelles
ma.masked_outside(arr, 0, 100)	Masque hors intervalle
ma.masked_inside(arr, 0, 100)	Masque dans l'intervalle
ma.masked_invalid(arr)	Masque NaN et infinis
ma.filled(arr_masqué, fill_value=0)	Remplace masqués par fill_value

OPÉRATIONS :

arr_ma.mask	Masque booléen
arr_ma.data	Données brutes (sans masque)
arr_ma.fill_value	Valeur de remplissage
arr_ma.compressed()	Données non-masquées (1D)
arr_ma.count()	Nombre de valeurs non-masquées
ma.mean(arr_ma)	Moyenne (ignore les masqués)
ma.sum(arr_ma)	Somme (ignore les masqués)

[np] Polynômes – numpy.polynomial

DESCRIPTION : Manipulation et ajustement de polynômes
 POURQUOI : Interpolation, régression polynomiale, analyse de signaux.
 CONTEXTE : Modélisation, lissage, calibration.

ANCIENNE API :

np.polyfit(x, y, deg=3)	Ajustement polynomiale degré 3
np.polyval(coeffs, x)	Évalue un polynôme
np.polyld(coeffs)	Objet polynôme
p = np.polyld(coeffs)	
p(x)	Évalue p en x
p.roots	Racines du polynôme
p.deriv()	Dérivée
p.integ()	Intégrale
np.roots(coeffs)	Racines du polynôme
np.polyder(coeffs)	Dérivée des coefficients
np.polyint(coeffs)	Intégrale des coefficients
np.polymul(p1, p2)	Produit de deux polynômes
np.polyadd(p1, p2)	Somme de deux polynômes
np.polysub(p1, p2)	Différence de deux polynômes
np.polydiv(p1, p2)	Division de deux polynômes

NOUVELLE API (polynomial.polynomial) :

```

from numpy.polynomial import polynomial as P
P.polyfit(x, y, deg=3)           Ajustement
P.polyval(x, coeffs)            Évaluation
# Polynômes de Legendre, Chebyshev, etc.
from numpy.polynomial import Chebyshev, Legendre, Hermite, Laguerre

```

[np] Tests et assertions – numpy.testing

DESCRIPTION : Fonctions de tests unitaires pour tableaux numpy
POURQUOI : Vérifier la correction des calculs numériques.
CONTEXTE : Tests unitaires, validation de refactoring.

FONCTIONS PRINCIPALES :

np.testing.assert_array_equal(a, b)	Égalité exacte élément par élément
np.testing.assert_array_almost_equal(a, b)	Égalité approchée (6 décimales)
np.testing.assert_array_almost_equal(a, b, decimal=4)	N décimales
np.testing.assert_allclose(a, b)	Égalité avec tolérance relative
np.testing.assert_allclose(a, b, rtol=1e-5, atol=1e-8)	
np.testing.assert_array_less(a, b)	a < b élément par élément
np.testing.assert_equal(a, b)	Égalité scalaire ou tableau
np.testing.assert_raises(Exception, func)	Vérification d'exception
np.testing.assert_warns(Warning, func)	Vérification d'avertissement
np.testing.assert_string_equal(a, b)	Égalité de chaînes

RÉCAPITULATIF DES FONCTIONS CLÉS PAR USAGE

USAGE	FONCTIONS PRINCIPALES
Créer un tableau	np.array, np.zeros, np.ones, np.arange, np.linspace
Forme / dimension	arr.shape, arr.ndim, arr.size, arr.dtype
Redimensionner	arr.reshape, arr.ravel, arr.flatten, np.squeeze
Ajouter une dimension	arr[np.newaxis], np.expand_dims
Transposer	arr.T, np.transpose, np.swapaxes, np.moveaxis
Concaténer	np.concatenate, np.vstack, np.hstack, np.stack
Diviser	np.split, np.array_split, np.vsplit, np.hsplit
Trier	np.sort, np.argsort, np.lexsort
Indexation avancée	arr[mask], arr[[i,j]], np.where, np.nonzero
Statistiques	np.mean, np.std, np.sum, np.min, np.max, np.median
NaN-safe	np.nanmean, np.nanstd, np.nansum, np.isnan
Algèbre linéaire	np.dot, @, np.linalg.solve, np.linalg.svd
Nombres aléatoires	default_rng, rng.normal, rng.integers, rng.choice
Maths élém.	np.sin, np.exp, np.log, np.sqrt, np.abs, np.clip
Arrondi	np.round, np.floor, np.ceil, np.trunc
Comparaison	np.allclose, np.isclose, np.array_equal
Valeurs uniques	np.unique, np.isin, np.intersect1d
Fourier	np.fft.rfft, np.fft.rfftfreq, np.fft.fftshift
Sauvegarde	np.save, np.load, np.savez, np.savetxt, np.loadtxt
Broadcasting	np.broadcast_to, np.einsum, np.outer
Polynômes	np.polyfit, np.polyval, np.polyld

— DTYPE QUICKREF —

TYPE	BITS	PLAGE / PRÉCISION	USAGE TYPIQUE
int8	8	-128 à 127	Images (si signé)
uint8	8	0 à 255	Images, pixels
int16	16	-32 768 à 32 767	Audio
int32	32	±2.1 milliards	Entiers généraux
int64	64	±9.2 × 10 ¹⁸	Grands entiers (défaut)
float16	16	~3 décimales	Deep learning (GPU)
float32	32	~7 décimales	ML, GPU
float64	64	~15 décimales (défaut)	Calcul scientifique
complex64	64	2 × float32	Signal
complex128	128	2 × float64 (défaut)	Calcul complexe
bool_	8	True / False	Masques

— NOTATION DES AXES —

Pour un tableau 2D (matrice) :
 axis=0 → opération par colonne (dans le sens des lignes, ↓)
 axis=1 → opération par ligne (dans le sens des colonnes, →)

Pour un tableau 3D (batch d'images) (N, H, W) :
 axis=0 → dans le sens des échantillons (N)
 axis=1 → dans le sens des hauteurs (H)
 axis=2 → dans le sens des largeurs (W)

axis=-1 → dernier axe (W ici)

=====